PROJE 20 / 20 · ÇALIŞAN TEMEL SÜRÜM

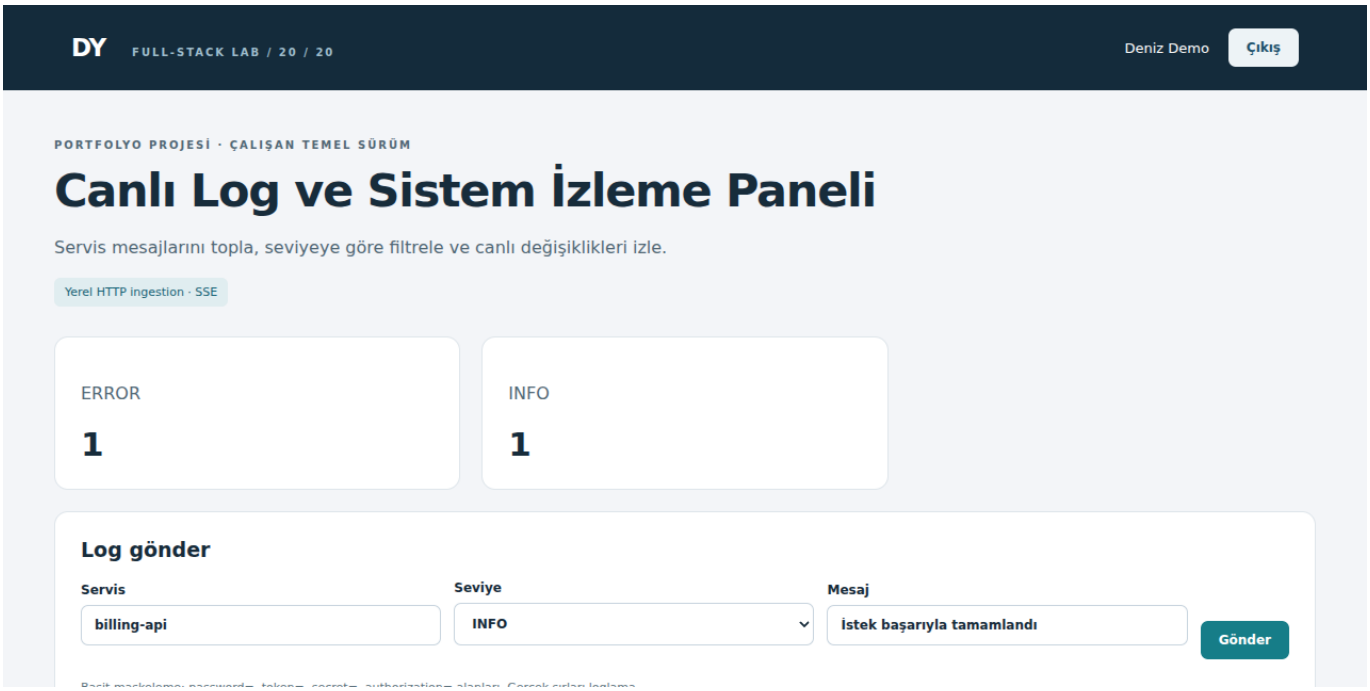
Canlı Log ve Sistem İzleme Paneli

Servis mesajlarını topla, seviyeye göre filtrele ve canlı değişiklikleri izle.

Yerel HTTP ingestion · SSE

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.



Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

INFO/WARN/ERROR kaydı · Servis ve zaman bilgisi · Temel sır maskeleme · Mesaj araması ve cursor · Seviye toplamları ve canlı yenileme

Bu sürümde olmayanlar

Gerçek uygulama agent'ı, distributed tracing ve full-text index yok. Arama LIKE kullanır. Kullanıcı başına son 10.000 log saklanır. Maskeleme basit desenlerle sınırlıdır; tüm kişisel verileri algılamaz.

02 / Backend ve veri modeli

Loglar yalnız oturum sahibine görünür. Ingestion alan uzunluklarını ve seviyeyi doğrular; belirli key=value sırlarını saklamadan önce maskeler. Liste id azalan sıralıdır; before parametresi eski sayfayı getirir. Yeni SSE olayı sayfayı en yeni 100 kayda döndürür. owner/level/id ve owner/service/id indeksleri bulunur.

Gerçek SQL şeması

```
CREATE TABLE logs(id INTEGER PRIMARY KEY,owner TEXT REFERENCES users(id),service
TEXT,level TEXT,message TEXT,at TEXT);

CREATE INDEX logs_filter ON logs(owner,level,id);

CREATE INDEX logs_service ON logs(owner,service,id);
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekilliği ve REFERENCES ilişkiyi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametreli SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
POST /api/logs	JSON: service, level, message	201
GET /api/logs	Query: level, q, before	200
GET /api/stats	Gövde yok	200
DELETE /api/logs	Gövde yok	200

Somut istek örneği

```
POST /api/logs
{"service":"billing","level":"ERROR","message":"token=abc failed"}
201 {"id":1}
```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error":"açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

• token= değeri saklanmadan önce maskelenmeli.

• Seviye ve mesaj filtresi doğru kayıtları getirmeli.

• İkinci hesap loglara erişememeli.

• Geçersiz seviye 422 dönmeli.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
await post('/logs',{service:'api',level:'ERROR',message:'token=secret-value
failed'},201);
await post('/logs',{service:'api',level:'INFO',message:'completed-unique'},201);
const logs=await get('/logs?level=ERROR');assert.equal(logs.length,1);assert.ok(!logs[0]
.message.includes('secret-value'));assert.ok(logs[0].message.includes('[REDACTED]'));
assert.equal((await get('/logs?q=completed-unique')).length,1);assert.equal((await
call('/api/logs','GET',undefined,200,other)).length,0);
await post('/logs',{service:'api',level:'INVALID',message:'bad'},422);
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

API entegrasyon testleri, üretim derlemesi ve Vue tip/şablon kontrolü uygulandı. Bunlar yük testi, penetrasyon testi veya tüm tarayıcılarda kapsamlı uçtan uca test yerine geçmez.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

INFO ve ERROR kayıtları ekle. token=demo-secret mesajının veritabanına REDACTED olarak yazıldığını kontrol et; başka hesabın listeyi göremediğini göster.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. Ayrı ingestion API anahtarı ekle
- 2. Structured JSON alanları ve FTS ekle
- 3. Zaman aralığı ve hata oranı grafikleri ekle
- 4. Batch ingestion ve retention job ekle

Anlamlı commit sırası

```
git add .
git commit -m "chore: add log-monitor baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

LIKE aramasının veri büyüdükçe maliyeti nedir? Hassas bilgiyi hiç loglamamak ile sonradan maskelemek arasındaki fark nedir?

Özel kurulum / ilerleme notu

Önce yerel sürümü tamamla. Sonraki aşamada PostgreSQL, migration, pagination ve gerçek kullanıcı yönetimini ihtiyaca göre ekle. Ortak kurulum ayrıntıları README.md ve başlangıç rehberindedir.